الحاويات والآلات في زمن الأزمات

Lecture 3 — Virtual Machines and Containers

> **Virtualization** : slicing one physical server into several isolated virtual machines. It is what makes cloud sharing possible.

Q/ Why virtualize ? (three reasons)

1. **Consolidation**: physical servers usually idle at low utilization; packing many VMs on one host raises utilization and cuts hardware, power and space.
2. **Isolation**: each VM has its own OS and resources; a crash or compromise in one should not reach its neighbours.
3. **Elasticity**: a VM is software — create, clone, resize or move it in minutes. This is the engine behind on-demand IaaS.

Hypervisors

> **Hypervisor (Virtual Machine Monitor)** : software that creates VMs and shares CPU, memory and I/O among them.

	Type 1 — bare metal	**Type 2 — hosted**
Runs on	Directly on the hardware	As an application on a host OS (Windows / Linux)
Examples	ESXi, Hyper-V, KVM, Xen	VMware Workstation, VirtualBox
Used in	Data centers / cloud providers	Desktops and labs

Cloud providers run **Type 1** (KVM-based or custom).

Q/ What are the approaches to virtualization ?

Approach	**How it works**	**Guest OS changed?**	**Trade-off**
Full virtualization	Hypervisor emulates the full hardware; **binary translation** for privileged instructions	No	Compatible, but slower

Approach	How it works	Guest OS changed?	Trade-off
Para-virtualization	Guest knows it is virtual and calls the hypervisor directly (**hypercalls**)	**Yes**	Fast, needs a modified OS
Hardware-assisted	CPU extensions (**Intel VT-x, AMD-V**) trap privileged operations in hardware	No	Fast and compatible — **today's default**

Lab check: if VT-x / AMD-V is **disabled in BIOS**, 64-bit guests may fail to start.

Q/ What does the hypervisor virtualize ?

1. **CPU**: schedules vCPUs onto physical cores. **Over-subscription** lets total vCPUs exceed cores (efficient, but can cause contention).
2. **Memory**: maps guest memory to host memory; **ballooning** and **page sharing** reclaim unused RAM.
3. **I/O**: presents virtual disks (often files on shared storage) and virtual NICs (connected to virtual switches).

Security consequence: **whoever controls the hypervisor controls every VM above it** — keep it minimal and patched.

Virtual machines

> **VM** : mostly **files** — virtual disks plus a configuration — which is why VMs are easy to copy and recover.

Q/ What is the VM lifecycle ?

1. **Create**: pick vCPU, RAM, disk, network; install or use an image.
2. **Run**: start, pause, suspend, resize.
3. **Snapshot**: save the state at a point in time, to roll back after a mistake or failed update.
4. **Clone**: copy a VM or a **golden template** to deploy many identical machines.
5. **Retire**: delete the VM and **securely wipe** its disks.

Security: snapshots and clones keep **old data and old secrets** (keys, passwords, unpatched software). Protect them like backups.

> **VM Migration** : moving a VM between hosts to balance load, patch hosts, or evacuate failing hardware.

Cold migration	Live migration
Shut the VM down, copy it, start it on the new host	Move a **running** VM with almost no downtime
Simple, but there is downtime	Users usually don't notice

Q/ What are the steps of live migration (pre-copy method) ?

1. Copy all memory pages while the VM keeps running.
2. Re-copy pages that changed (repeat).
3. Pause briefly; send the last pages and CPU state.
4. Resume on the new host.

Memory travels over the network: **encrypt the migration channel**.

Containers

> **Container** : an ordinary process on the **host kernel**, wrapped by Linux features that limit what it can see and use. No guest OS.

> **Namespaces** : control what a container can **SEE** — separate views of processes (PID), network, mounts, hostname and users.

> **cgroups (control groups)** : control what a container can **USE** — limits on CPU, memory and I/O so one container can't starve the others.

Because there is no guest OS, a container starts in **seconds**, uses **megabytes**, and you can pack many more per host. The price is a **shared kernel**.

Aspect	Virtual machine	Container
Isolation	**Strong**: separate kernel per VM	**Weaker**: shared host kernel
Startup	Minutes	Seconds
Size	Gigabytes	Megabytes

Aspect	Virtual machine	Container
Density per host	Tens	Hundreds
Guest OS	Any (Windows, Linux…)	Same kernel family as the host
Stack	App → Guest OS → Hypervisor → Hardware	App → Container engine → Host kernel → Hardware
Best for	Legacy apps, strong tenant separation	Microservices, CI/CD, scale-out

In practice they combine: **containers usually run inside VMs** in the cloud.

Docker

> **Docker** : packages an application and everything it needs, so it runs the same on a laptop, a lab server or a cloud VM.

Q/ What are Docker's components ?

1. **Dockerfile**: a text recipe — base image, files, commands.
2. **Image**: a read-only, **layered** template built from the Dockerfile.
3. **Container**: a running instance of an image, with a thin writable layer.
4. **Engine (daemon)**: builds images and runs containers on the host.
5. **Registry**: stores and distributes images (Docker Hub, private registry).
6. **Volume / Network**: persistent data outside the container; virtual networks between containers.

Q/ What is the Docker workflow ?

- **Build → Ship → Run**: Dockerfile — `docker build` → Image — `docker push` → Registry — `docker pull` / `docker run` → Container.

```
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/
docker build -t mysite .
docker run -d -p 8080:80 mysite
```

`-p 8080:80` maps **host port 8080** to **container port 80**.

Security lens

VM risks	Container risks
VM escape through a hypervisor flaw	**Kernel exploit** reaches every container
Side-channel attacks between co-located VMs	**Privileged** containers
Stale snapshots	Vulnerable or untrusted images
VM sprawl	**Secrets baked into images**

Q/ What are the good security practices ?

1. Patch the hypervisor and host kernel; keep them minimal.
2. Use trusted, scanned images; pin versions; **never store secrets in an image**.
3. Don't run containers as **root** or with `--privileged`; drop capabilities.
4. Separate sensitive workloads on different hosts or inside VMs.

Q/ Mini case — VM or container ?

1. Old Windows-only grading system → **VM** (needs another OS).
2. Student portal with registration spikes → **containers** (fast scale-out).
3. Lab where outsiders run untrusted code → **VM** (strong isolation).

- They can be combined: containers inside VMs.

Summary of Cloud Computing L3: Virtual Machines & Containers. Built only from the lecture file; check the original slides for anything you're unsure about.